

Piles génériques et validation d'expressions mathématiques en C++

Table des matières

1	Introduction	2
2	Piles génériques	2
2.1	Problématique	2
2.2	Pile à tableau dynamique	2
2.3	Pile à liste chaînée	2
2.4	Tests fonctionnels	3
2.5	Comparaison des représentations	3
3	Mesure des opérations des piles	3
3.1	Protocole	3
3.2	Résultats et interprétation	3
4	Validation d'expressions mathématiques	4
4.1	Problématique	4
4.2	Analyse lexicale et multiplications implicites	4
4.3	Vérification syntaxique et ordre des opérations	4
4.4	Exemple de résultat	5
5	Interface DLL et programme client	5
5.1	Fonction exportée	5
5.2	Fonctionnement du client	5
5.3	Compilation	5
6	Résultats et conclusion	6

1 Introduction

Ce travail présente deux implémentations d'une pile générique en C++ et leur application à l'analyse d'expressions mathématiques. La pile est une structure de données de type LIFO (*Last In, First Out*) : le dernier élément ajouté est le premier retiré. Les deux représentations étudiées sont le tableau dynamique et la liste chaînée.

Le programme de validation lit une expression, reconnaît ses éléments, vérifie sa syntaxe et présente les opérations symboliques dans leur ordre de traitement. Une interface sous forme de bibliothèque dynamique (DLL) permet également d'appeler cette validation depuis un programme client. Des mesures de durée sont affichées pour les opérations sur les piles et pour la validation d'une expression.

2 Piles génériques

2.1 Problématique

L'objectif est de construire une structure de pile réutilisable avec plusieurs types de données. Elle doit notamment offrir l'ajout et le retrait d'un élément, la consultation du sommet, la vérification de l'état vide et la lecture de la taille. Il faut également traiter explicitement les tentatives de consultation ou de retrait lorsque la pile ne contient aucun élément.

2.2 Pile à tableau dynamique

Dans `mainTDV.cpp`, la classe générique `Stack<T>` s'appuie sur `std::vector<T>`. Le sommet est le dernier élément du vecteur.

- `push` ajoute une valeur à la fin du vecteur.
- `pop` renvoie le dernier élément puis le supprime.
- `top` renvoie une copie du sommet sans le retirer.
- `isEmpty` indique si le vecteur ne contient aucun élément.
- `size` renvoie le nombre d'éléments et `getCapacity` sa capacité réservée.
- `clear` supprime les éléments tout en conservant la capacité allouée.

Le constructeur réserve initialement une capacité de deux éléments. Lorsque cette capacité est dépassée, le vecteur gère lui-même son agrandissement. Si `pop` ou `top` est appelé sur une pile vide, un message est affiché et la méthode renvoie une valeur construite par défaut avec `T()`.

2.3 Pile à liste chaînée

Dans `mainLC.cpp`, la classe générique `Pile<T>` stocke les éléments dans des maillons. Chaque maillon contient une valeur et un pointeur vers le maillon suivant. Le membre `tete\_` désigne le sommet de la pile.

Les maillons sont gérés par `std::unique\_ptr`, qui exprime une propriété unique et libère automatiquement les ressources. `empiler` ajoute un maillon en tête. `depiler` récupère la valeur de la tête, puis transfère la tête vers le maillon suivant. La méthode `sommet` consulte la valeur au sommet, `taille` renvoie le nombre de maillons et `estVide` vérifie si la tête est nulle.

La copie de cette pile est interdite. Les méthodes `depiler` et `sommet` lancent `std::underflow\_error` si la pile est vide. La méthode `vider` détruit les maillons un par un et remet le compteur à zéro. Le destructeur appelle cette méthode afin de libérer les éléments restants.

2.4 Tests fonctionnels

Les tests de `mainTDV.cpp` vérifient l'ajout de trois entiers, le sommet, l'ordre LIFO, la taille, la capacité, la méthode `clear` et le comportement de `pop` et `top` sur une pile vide.

Les tests de `mainLC.cpp` valident l'ordre LIFO avec des entiers, l'emploi de la pile avec des chaînes de caractères, la remise à zéro par `vider` et la levée d'une exception lors du dépilement d'une pile vide.

2.5 Comparaison des représentations

Critère	Tableau dynamique	Liste chaînée
Stockage	Éléments contigus gérés par <code>std::vector<T></code> .	Maillons reliés par des pointeurs.
Accès au sommet	Dernier élément du vecteur.	Premier maillon, désigné par la tête.
Ajout et retrait	À la fin du vecteur.	À la tête de la liste.
Mémoire	Capacité éventuellement supérieure au nombre d'éléments.	Allocation d'un maillon pour chaque élément.
Pile vide	Message et valeur par défaut.	Exception <code>std::underflow_error</code> .

3 Mesure des opérations des piles

3.1 Protocole

Les deux programmes effectuent la même séquence de mesure avec 1 000 entiers. Le chronomètre de `std::chrono::steady\_clock` démarre avant les empilements. Le programme empile les valeurs de 0 à 999, puis dépile 1 000 valeurs. Le chronomètre s'arrête après le dernier dépilement. La durée totale des deux phases est affichée en microsecondes.

La somme des valeurs dépilées est également calculée. Une assertion vérifie qu'elle vaut 499 500, ce qui confirme que les 1 000 éléments attendus ont été retirés. Le protocole est présent dans `mainTDV.cpp` et `mainLC.cpp`.

3.2 Résultats et interprétation

À l'exécution, chaque programme affiche une mesure sous la forme suivante :

```

1 Tableau dynamique - 1000 empilements + 1000 depilements : ... us
2 Liste chainee - 1000 empilements + 1000 depilements : ... us
```

La durée numérique dépend de la machine, du compilateur, des options de compilation et de la charge du système. Elle doit donc être relevée lors de l'exécution et ne constitue pas une constante à inscrire à l'avance dans le rapport. Cette mesure porte sur une seule

exécution et donne une indication expérimentale, non une comparaison exhaustive des performances.

Les deux structures effectuent l'ajout et le retrait au sommet. Le tableau dynamique peut parfois devoir réallouer sa capacité lors d'un ajout, tandis que la liste chaînée crée un maillon pour chaque nouvel élément. Le résultat mesuré reflète le coût global de la séquence choisie, dans les conditions locales d'exécution.

4 Validation d'expressions mathématiques

4.1 Problématique

Le fichier `verification\_validite.cpp` traite une expression saisie sous forme de texte. Il doit identifier les nombres, les variables, les opérateurs et les parenthèses, détecter les erreurs de syntaxe et établir l'ordre des opérations. Il ne réalise pas de calcul numérique : les résultats présentés sont des étapes symboliques.

4.2 Analyse lexicale et multiplications implicites

La fonction `analyserJetons` parcourt l'expression et la transforme en jetons typés : nombres, variables, opérateurs, parenthèses ouvrantes ou parenthèses fermantes. Les espaces sont ignorés. Les nombres peuvent comporter un point décimal, et les noms de variables peuvent commencer par une lettre ou un underscore puis contenir des lettres, des chiffres et des underscores. Un caractère non reconnu ou un point isolé provoque une erreur.

Une seconde étape insère une multiplication dans certaines juxtapositions usuelles : $5x$ devient $5*x$, $2(x+1)$ devient $2*(x+1)$, et $(x+1)(x-1)$ devient $(x+1)*(x-1)$. Deux nombres directement juxtaposés ne sont pas interprétés comme une multiplication.

4.3 Vérification syntaxique et ordre des opérations

La fonction `verifierExpression` utilise deux piles : l'une conserve les opérateurs en attente et les parenthèses ouvrantes, l'autre conserve les valeurs et les sous-expressions symboliques. Une liste recueille les opérations au moment où elles sont appliquées.

Les priorités codées sont les suivantes :

Opérateurs	Priorité	Associativité
Exponentiation <code>\string ^</code>	4	À droite
Signes unaires <code>u+</code> , <code>u-</code>	3	À droite
Multiplication et division	2	À gauche
Addition et soustraction	1	À gauche

Les signes `+` et `-` peuvent être unaires au début d'une expression ou après un autre opérateur. Ils sont alors représentés en interne par `u+` et `u-`. L'algorithme traite les opérateurs selon leur priorité et leur associativité, et termine les opérations d'un groupe lorsqu'il rencontre la parenthèse fermante correspondante.

La validation détecte notamment une expression vide, un opérateur sans opérande, deux valeurs sans opérateur, une parenthèse vide, des parenthèses non équilibrées et une expression se terminant par un opérateur. Une expression complète doit finalement laisser une seule sous-expression sur la pile.

4.4 Exemple de résultat

Pour $2+3*x$, la multiplication est traitée avant l'addition. Le programme affiche les étapes symboliques :

```
1 1. 3 * x
2 2. 2 + (3 * x)
```

Pour $2(x+1)$, la multiplication implicite est insérée et l'expression entre parenthèses est traitée avant la multiplication par 2. Une expression invalide provoque l'affichage d'un message indiquant la cause de l'échec.

5 Interface DLL et programme client

5.1 Fonction exportée

L'interface déclarée dans `verification\_validite.h` expose la fonction C `verifier\_valider\_ex`. La convention `extern "C"` rend le symbole callable depuis un programme client, tandis que la macro `VERIFICATION\_API` gère l'exportation ou l'importation de la fonction sous Windows.

La fonction reçoit l'expression, un tampon destiné au message, la capacité de ce tampon et un pointeur où écrire la taille requise. Cette taille comprend le caractère nul final. Le paramètre de sortie peut être nul lors d'un appel destiné à connaître la taille nécessaire.

Statut	Signification
1	L'expression est valide et le message de résultat a été écrit dans le tampon.
0	L'expression est invalide et le message d'erreur a été écrit dans le tampon.
-1	La taille du tampon est insuffisante, le tampon de sortie est nul ou le pointeur de taille requise est nul. La taille requise est fournie lorsqu'il est possible de l'écrire.

5.2 Fonctionnement du client

Le programme `client\_verification.cpp` lit une expression depuis la console et appelle la fonction exportée. Il commence avec un tampon de 256 caractères. Si le statut indique que le résultat ne tient pas dans le tampon et que la taille requise est supérieure à sa capacité, le client redimensionne le tampon puis rappelle la fonction. Il affiche ensuite le statut, le message de validation ou d'erreur et la durée mesurée en microsecondes.

Le chronométrage du client couvre l'appel de validation et, lorsqu'un agrandissement est nécessaire, le second appel. La durée affichée dépend donc de l'expression testée et de l'environnement d'exécution.

5.3 Compilation

La compilation transforme le code source C++ en programme exécutable. Elle se déroule conceptuellement en plusieurs étapes :

1. **Prétraitement** : les directives telles que `##include` sont traitées et le contenu des en-têtes nécessaires est intégré au fichier source.
2. **Compilation** : le code C++ prétraité est analysé et traduit en instructions assembleur. Le compilateur signale à cette étape les erreurs de syntaxe et certains avertissements.
3. **Assemblage** : le code assembleur est converti en code machine dans un fichier objet, généralement avec l'extension `.o`.
4. **Édition de liens** : les fichiers objets et les bibliothèques nécessaires sont réunis pour produire un exécutable ou une bibliothèque dynamique. Cette étape résout les références entre fonctions et bibliothèques.

Le compilateur `g++` peut réaliser toutes ces étapes en une seule commande. Par exemple, les deux piles peuvent être compilées séparément en exécutables :

```
1 g++ -std=c++17 -O2 -Wall -Wextra -pedantic mainTDV.cpp -o mainTDV.exe
2 g++ -std=c++17 -O2 -Wall -Wextra -pedantic mainLC.cpp -o mainLC.exe
3
```

L'option `-std=c++17` sélectionne la norme C++17, `-O2` active des optimisations, et `-Wall -Wextra -pedantic` demande des avertissements supplémentaires. L'option `-o` indique le nom du fichier produit. Pour séparer la génération du fichier objet et l'édition de liens, on peut utiliser `-c` :

```
1 g++ -std=c++17 -O2 -Wall -Wextra -pedantic -c mainTDV.cpp -o mainTDV.o
2 g++ mainTDV.o -o mainTDV.exe
3
```

La première commande produit le fichier objet sans créer l'exécutable. La seconde lie ce fichier pour obtenir le programme exécutable. Dans le cas de la validation d'expressions, la construction est répartie entre la bibliothèque DLL qui contient la fonction de validation et le programme client qui l'utilise.

Le script `build\_dll.bat` vérifie que `g++` est disponible, construit la DLL et sa bibliothèque d'importation avec les options C++17, puis compile le client en le reliant à la bibliothèque :

```
1 g++ -std=c++17 -O2 -Wall -Wextra -pedantic -DVERIFICATION_DLL_BUILD -shared
   verification_validite.cpp -o verification_validite.dll -Wl,--out-implib,
   libverification_validite.dll.a
2
3 g++ -std=c++17 -O2 -Wall -Wextra -pedantic client_verification.cpp -L. -
   lverification_validite -o client_verification.exe
```

La compilation nécessite un compilateur GCC accessible dans le chemin système. Le client doit également pouvoir charger la DLL au moment de son exécution.

6 Résultats et conclusion

Les tests fonctionnels vérifient le principe LIFO, la consultation et le retrait du sommet, les états vide et non vide, la taille, la capacité du tableau dynamique, la suppression des éléments et les comportements d'erreur définis par chaque implémentation. Le programme d'analyse vérifie les jetons et la syntaxe, respecte les priorités et les parenthèses, puis fournit un ordre symbolique des opérations sans effectuer le calcul.

Les mesures chronométrées rendent visible le temps d'une séquence de 1 000 empilements suivie de 1 000 dépilements, ainsi que celui de la validation appelée par le client. Ces résultats sont propres à chaque exécution. Enfin, la DLL sépare la logique de validation de son programme client et communique le résultat au moyen d'un statut et d'un tampon de sortie.